

Quick demonstration of performance gains in variance estimation

Peio Alcorta and Petr Mariel

March 2025

```
# R code to calculate using analytical solutions the log-likelihood, gradient, Hessian,  
# robust and non-robust variance estimates, and asymptotic variance of MNL-based LCM.
```

```
# Credit: Peio Alcorta (https://orcid.org/0000-0002-0625-7777)  
# and Petr Mariel (https://orcid.org/0000-0002-7412-0684).
```

```
# Available at: https://github.com/discretechoice/LCM.
```

```
# The formulas will be made publicly available after publication.
```

```
# Preliminaries
```

```
rm(list = ls(all = TRUE))          # Clear workspace  
Sys.setenv(LANG = "en")           # Set language  
setwd(dirname(rstudioapi::getSourceEditorContext()$path)) # Set working directory
```

```
set.seed(111)
```

```
# Loading R packages
```

```
library(evd)  
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

```
library(matrixStats)
```

```
##
```

```
## Attaching package: 'matrixStats'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
## count
```

[illegible]

2

```

database$x_alt2_attr4 <- sample(c(0,1),size=n_rows, replace=TRUE, prob=c(0.5, 0.5))

database$x_alt1_attr5 <- sample(c(0,1),size=n_rows, replace=TRUE, prob=c(0.5, 0.5))
database$x_alt2_attr5 <- sample(c(0,1),size=n_rows, replace=TRUE, prob=c(0.5, 0.5))

# Arrange the data
database <- database %>% arrange(id_individual, num_choice_card)

# Create class allocation variables

# z_1 equal to 1 (constant)
database$z_1 <- 1

# z_2 and z_3 are continuous and distributed U(0,1)
database$z_2 <- rep(runif(n_individuals,0,1),each=n_choices)
database$z_3 <- rep(runif(n_individuals,0,1),each=n_choices)

# z_4 and z_5 are dummy with 50% of 1s
database$z_4 <- rep(sample(c(0,1),size=n_individuals,replace=T,prob=c(0.5, 0.5)),each=n_choices)
database$z_5 <- rep(sample(c(0,1),size=n_individuals,replace=T,prob=c(0.5, 0.5)),each=n_choices)

# Attribute parameters
beta_class1_attr1 <- 1.0
beta_class1_attr2 <- 1.5
beta_class1_attr3 <- -2.0
beta_class1_attr4 <- 1.2
beta_class1_attr5 <- 0.2

beta_class2_attr1 <- 0.5
beta_class2_attr2 <- 0.5
beta_class2_attr3 <- -1.5
beta_class2_attr4 <- -1.0
beta_class2_attr5 <- -0.7

beta_class3_attr1 <- -1.0
beta_class3_attr2 <- -1.5
beta_class3_attr3 <- 2.0
beta_class3_attr4 <- 1.2
beta_class3_attr5 <- 0.2

beta_class4_attr1 <- -0.5
beta_class4_attr2 <- -0.5
beta_class4_attr3 <- -1.5
beta_class4_attr4 <- 1.0
beta_class4_attr5 <- 0.7

# Latent class membership parameters
gamma1_c2 <- -0.5
gamma2_c2 <- 1.0
gamma3_c2 <- 1.5
gamma4_c2 <- -1.0
gamma5_c2 <- 0.7

```

```

gamma1_c3      <- 1.5
gamma2_c3      <- -2.0
gamma3_c3      <- -1.5
gamma4_c3      <- 3.0
gamma5_c3      <- -1.0

gamma1_c4      <- 0.5
gamma2_c4      <- 0.2
gamma3_c4      <- -1.0
gamma4_c4      <- -1.0
gamma5_c4      <- -0.7

# Generation of class allocation probabilities
database$latent_class2 <- (
  gamma1_c2
+ gamma2_c2 * database$z_2
+ gamma3_c2 * database$z_3
+ gamma4_c2 * database$z_4
+ gamma5_c2 * database$z_5
+ rep(rlogis(n_individuals,0,1),each=n_choices)
)

database$latent_class3 <- (
  gamma1_c3
+ gamma2_c3 * database$z_2
+ gamma3_c3 * database$z_3
+ gamma4_c3 * database$z_4
+ gamma5_c3 * database$z_5
+ rep(rlogis(n_individuals,0,1),each=n_choices)
)

database$latent_class4 <- (
  gamma1_c4
+ gamma2_c4 * database$z_2
+ gamma3_c4 * database$z_3
+ gamma4_c4 * database$z_4
+ gamma5_c4 * database$z_5
+ rep(rlogis(n_individuals,0,1),each=n_choices)
)

database$id_class <- 1
database$id_class[database$latent_class2 > 0] <- 2
database$id_class[database$latent_class3 > database$latent_class2] <- 3
database$id_class[database$latent_class4 > database$latent_class3] <- 4

# Generation of utilities for each class
# Random errors generation Gumbel(0,1) for each alternative's utility
database$error1 <- rgumbel(n_rows ,loc=0, scale=1)
database$error2 <- rgumbel(n_rows ,loc=0, scale=1)

# CLASS 1
database$utility1_class1 <- (
  # Creating utility 1
  beta_class1_attr1 * database$x_alt1_attr1
+ beta_class1_attr2 * database$x_alt1_attr2

```

```

+ beta_class1_attr3 * database$x_alt1_attr3
+ beta_class1_attr4 * database$x_alt1_attr4
+ beta_class1_attr5 * database$x_alt1_attr5
+
)

database$utility2_class1 <- (
    # Creating utility 2
    beta_class1_attr1 * database$x_alt2_attr1
+ beta_class1_attr2 * database$x_alt2_attr2
+ beta_class1_attr3 * database$x_alt2_attr3
+ beta_class1_attr4 * database$x_alt2_attr4
+ beta_class1_attr5 * database$x_alt2_attr5
+
    database$error2
)

# CLASS 2
database$utility1_class2 <- (
    # Creating utility 1
    beta_class2_attr1 * database$x_alt1_attr1
+ beta_class2_attr2 * database$x_alt1_attr2
+ beta_class2_attr3 * database$x_alt1_attr3
+ beta_class2_attr4 * database$x_alt1_attr4
+ beta_class2_attr5 * database$x_alt1_attr5
+
    database$error1
)

database$utility2_class2 <- (
    # Creating utility 2
    beta_class2_attr1 * database$x_alt2_attr1
+ beta_class2_attr2 * database$x_alt2_attr2
+ beta_class2_attr3 * database$x_alt2_attr3
+ beta_class2_attr4 * database$x_alt2_attr4
+ beta_class2_attr5 * database$x_alt2_attr5
+
    database$error2
)

# CLASS 3
database$utility1_class3 <- (
    # Creating utility 1
    beta_class3_attr1 * database$x_alt1_attr1
+ beta_class3_attr2 * database$x_alt1_attr2
+ beta_class3_attr3 * database$x_alt1_attr3
+ beta_class3_attr4 * database$x_alt1_attr4
+ beta_class3_attr5 * database$x_alt1_attr5
+
    database$error1
)

database$utility2_class3 <- (
    # Creating utility 2
    beta_class3_attr1 * database$x_alt2_attr1
+ beta_class3_attr2 * database$x_alt2_attr2
+ beta_class3_attr3 * database$x_alt2_attr3
+ beta_class3_attr4 * database$x_alt2_attr4
+ beta_class3_attr5 * database$x_alt2_attr5
+
    database$error2
)

```

```

# CLASS 4
database$utility1_class4 <- (
    # Creating utility 1
    beta_class4_attr1 * database$x_alt1_attr1
  + beta_class4_attr2 * database$x_alt1_attr2
  + beta_class4_attr3 * database$x_alt1_attr3
  + beta_class4_attr4 * database$x_alt1_attr4
  + beta_class4_attr5 * database$x_alt1_attr5
  + database$error1
)

database$utility2_class4 <- (
    # Creating utility 2
    beta_class4_attr1 * database$x_alt2_attr1
  + beta_class4_attr2 * database$x_alt2_attr2
  + beta_class4_attr3 * database$x_alt2_attr3
  + beta_class4_attr4 * database$x_alt2_attr4
  + beta_class4_attr5 * database$x_alt2_attr5
  + database$error2
)

# Generate choice depending on which class an individual belongs to
for (j in 1:n_rows) {

  if(database$id_class[j] == 1){
    database$choice[j] <- which.max(c(database$utility1_class1[j],
                                      database$utility2_class1[j])
    )
  }
  if(database$id_class[j] == 2){
    database$choice[j] <- which.max(c(database$utility1_class2[j],
                                      database$utility2_class2[j])
    )
  }
  if(database$id_class[j] == 3){
    database$choice[j] <- which.max(c(database$utility1_class3[j],
                                      database$utility2_class3[j])
    )
  }
  if(database$id_class[j] == 4){
    database$choice[j] <- which.max(c(database$utility1_class4[j],
                                      database$utility2_class4[j])
    )
  }
}

#####
# MODEL ESTIMATION USING APOLLO
#####

```

```

apollo_initialise()
# Initialise code
apollo_control = list(
  modelName      = "4Q"           ,
  modelDescr     = "LCM"          ,
  indivID        = "id_individual" ,
  nCores         = 1              ,
  panelData      = TRUE
)

# Vector of parameters
apollo_beta=c(cl1_b_attr1        = beta_class1_attr1,
              cl1_b_attr2        = beta_class1_attr2,
              cl1_b_attr3        = beta_class1_attr3,
              cl1_b_attr4        = beta_class1_attr4,
              cl1_b_attr5        = beta_class1_attr5,
              #
              cl2_b_attr1        = beta_class2_attr1,
              cl2_b_attr2        = beta_class2_attr2,
              cl2_b_attr3        = beta_class2_attr3,
              cl2_b_attr4        = beta_class2_attr4,
              cl2_b_attr5        = beta_class2_attr5,
              #
              cl3_b_attr1        = beta_class3_attr1,
              cl3_b_attr2        = beta_class3_attr2,
              cl3_b_attr3        = beta_class3_attr3,
              cl3_b_attr4        = beta_class3_attr4,
              cl3_b_attr5        = beta_class3_attr5,
              #
              cl4_b_attr1        = beta_class4_attr1,
              cl4_b_attr2        = beta_class4_attr2,
              cl4_b_attr3        = beta_class4_attr3,
              cl4_b_attr4        = beta_class4_attr4,
              cl4_b_attr5        = beta_class4_attr5,
              #
              cl2_b_z1           = gamma1_c2,
              cl2_b_z2           = gamma2_c2,
              cl2_b_z3           = gamma3_c2,
              cl2_b_z4           = gamma4_c2,
              cl2_b_z5           = gamma5_c2,
              #
              cl3_b_z1           = gamma1_c3,
              cl3_b_z2           = gamma2_c3,
              cl3_b_z3           = gamma3_c3,
              cl3_b_z4           = gamma4_c3,
              cl3_b_z5           = gamma5_c3,
              #
              cl4_b_z1           = gamma1_c4,
              cl4_b_z2           = gamma2_c4,
              cl4_b_z3           = gamma3_c4,
              cl4_b_z4           = gamma4_c4,
              cl4_b_z5           = gamma5_c4
)

```

```

apollo_fixed =c()

# Latent class components

apollo_lcPars=function(apollo_beta, apollo_inputs){
  lcpars = list()
  lcpars[["b_attr1"]] = list(c11_b_attr1 ,c12_b_attr1 , c13_b_attr1 , c14_b_attr1)
  lcpars[["b_attr2"]] = list(c11_b_attr2 ,c12_b_attr2 , c13_b_attr2 , c14_b_attr2)
  lcpars[["b_attr3"]] = list(c11_b_attr3 ,c12_b_attr3 , c13_b_attr3 , c14_b_attr3)
  lcpars[["b_attr4"]] = list(c11_b_attr4 ,c12_b_attr4 , c13_b_attr4 , c14_b_attr4)
  lcpars[["b_attr5"]] = list(c11_b_attr5 ,c12_b_attr5 , c13_b_attr5 , c14_b_attr5)

  # Utilities (probability logit parameters) of class allocation model
  V=list()
  V[["class_a"]] = 0.0
  V[["class_b"]] = c12_b_z1*z_1 + c12_b_z2*z_2 + c12_b_z3*z_3 + c12_b_z4*z_4 + c12_b_z5*z_5
  V[["class_c"]] = c13_b_z1*z_1 + c13_b_z2*z_2 + c13_b_z3*z_3 + c13_b_z4*z_4 + c13_b_z5*z_5
  V[["class_d"]] = c14_b_z1*z_1 + c14_b_z2*z_2 + c14_b_z3*z_3 + c14_b_z4*z_4 + c14_b_z5*z_5

  # Settings for class allocation models
  classAlloc_settings = list(
    classes      = c(class_a=1, class_b=2, class_c=3,class_d=4),
    utilities     = V
  )
  lcpars[["pi_values"]] = apollo_classAlloc(classAlloc_settings)
  return(lcpars)
}

apollo_inputs = apollo_validateInputs()

```

```

## All checks on apollo_control completed.
## All checks on database completed.

```

```

apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){

  # Attach inputs and detach after function exit
  apollo_attach(apollo_beta, apollo_inputs)
  on.exit(apollo_detach(apollo_beta, apollo_inputs))

  # Create list of probabilities P
  P = list()

  # Define settings for MNL model component that are generic across classes
  mnl_settings = list(
    alternatives = c(alt1=1, alt2=2),
    avail       = list(alt1=1, alt2=1),
    choiceVar   = choice
  )

  S <- 4
  # Loop over classes
  for(s in 1:S){
    # Compute class-specific utilities
    V=list()

```



```

V[["alt1"]] = (b_attr1[[s]]*x_alt1_attr1 +
               b_attr2[[s]]*x_alt1_attr2 +
               b_attr3[[s]]*x_alt1_attr3 +
               b_attr4[[s]]*x_alt1_attr4 +
               b_attr5[[s]]*x_alt1_attr5
               )
V[["alt2"]] = (b_attr1[[s]]*x_alt2_attr1 +
               b_attr2[[s]]*x_alt2_attr2 +
               b_attr3[[s]]*x_alt2_attr3 +
               b_attr4[[s]]*x_alt2_attr4 +
               b_attr5[[s]]*x_alt2_attr5
               )

mnl_settings$utilities      = V
mnl_settings$componentName = paste0("Class_",s)

# Compute within-class choice probabilities using MNL model
P[[paste0("Class_",s)]] = apollo_mnl(mnl_settings, functionality)

# Take product across observation for same individual
P[[paste0("Class_",s)]] = apollo_panelProd(P[[paste0("Class_",s)]] ,
                                           apollo_inputs,
                                           functionality
                                           )

}

# Compute latent class model probabilities
lc_settings = list(inClassProb = P, classProb=pi_values)
P[["model"]] = apollo_lc(lc_settings, apollo_inputs, functionality)

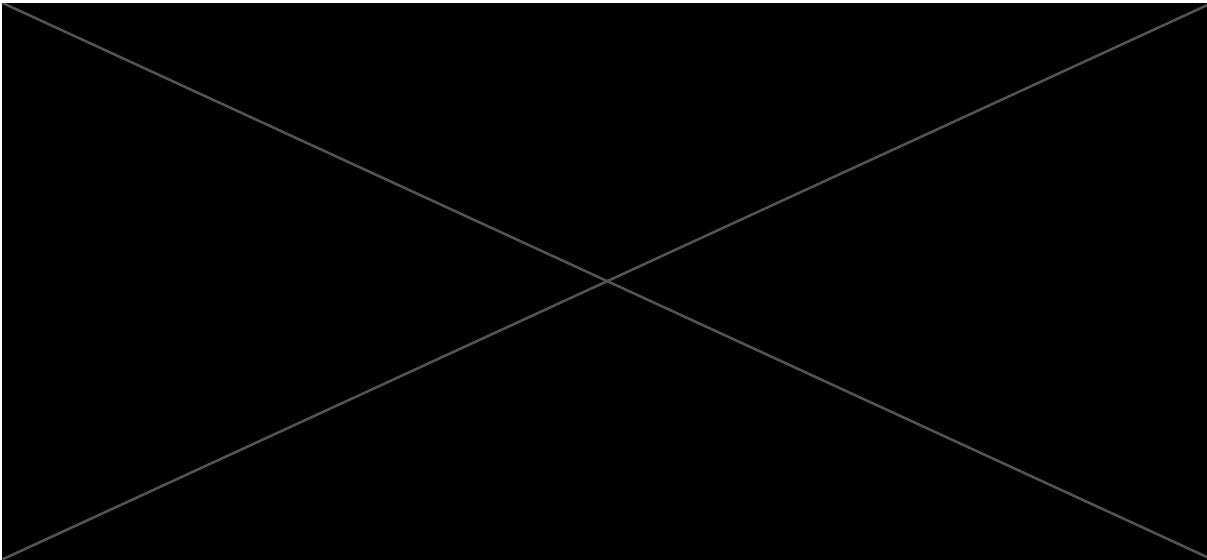
# Prepare and return outputs of function
P = apollo_prepareProb(P, apollo_inputs, functionality)
return(P)
}

# Estimate the model
model = apollo_estimate(apollo_beta,
                        apollo_fixed,
                        apollo_probabilities,
                        apollo_inputs,
                        estimate_settings = list(writeIter      = FALSE,
                                                  silent          = TRUE ,
                                                  maxIterations   = 100000)
                        )

#####
#   VARIANCE ESTIMATION USING ANALYTICAL FORMULAS
#####

# These formulas will be made publicly available after publication

```



The analytical solutions are evaluated at the ML estimate to which Apollo converged.

```
theta <- list( "beta"= list( "class_1"=c(model[["estimate"]][["cl1_b_attr1"]],  
                                         model[["estimate"]][["cl1_b_attr2"]],  
                                         model[["estimate"]][["cl1_b_attr3"]],  
                                         model[["estimate"]][["cl1_b_attr4"]],  
                                         model[["estimate"]][["cl1_b_attr5"]]  
                                         ),  
              "class_2"=c(model[["estimate"]][["cl2_b_attr1"]],  
                           model[["estimate"]][["cl2_b_attr2"]],  
                           model[["estimate"]][["cl2_b_attr3"]],  
                           model[["estimate"]][["cl2_b_attr4"]],  
                           model[["estimate"]][["cl2_b_attr5"]]  
                           ),  
              "class_3"=c(model[["estimate"]][["cl3_b_attr1"]],  
                           model[["estimate"]][["cl3_b_attr2"]],  
                           model[["estimate"]][["cl3_b_attr3"]],  
                           model[["estimate"]][["cl3_b_attr4"]],  
                           model[["estimate"]][["cl3_b_attr5"]]  
                           ),  
              "class_4"=c(model[["estimate"]][["cl4_b_attr1"]],  
                           model[["estimate"]][["cl4_b_attr2"]],  
                           model[["estimate"]][["cl4_b_attr3"]],  
                           model[["estimate"]][["cl4_b_attr4"]],  
                           model[["estimate"]][["cl4_b_attr5"]]  
                           )  
              ),  
  "gamma" = list("class_1"=c(0,0,0,0,0),  
                 "class_2"=c(model[["estimate"]][["cl2_b_z1"]],  
                             model[["estimate"]][["cl2_b_z2"]],  
                             model[["estimate"]][["cl2_b_z3"]],  
                             model[["estimate"]][["cl2_b_z4"]],  
                             model[["estimate"]][["cl2_b_z5"]]  
                             ),  
                 "class_3"=c(model[["estimate"]][["cl3_b_z1"]],  
                             model[["estimate"]][["cl3_b_z2"]],  
                             model[["estimate"]][["cl3_b_z3"]],  
                             model[["estimate"]][["cl3_b_z4"]],
```

```

        model[["estimate"]][["cl3_b_z5"]]
      ),
      "class_4"=c(model[["estimate"]][["cl4_b_z1"]],
        model[["estimate"]][["cl4_b_z2"]],
        model[["estimate"]][["cl4_b_z3"]],
        model[["estimate"]][["cl4_b_z4"]],
        model[["estimate"]][["cl4_b_z5"]]
      )
    )
  )

# We measure the computation time for standard errors and robust standard errors,
# encompassing intermediate calculations such as the Hessian, its inverse, the OPG estimator,
# and the robust sandwich variance estimator.

t1 <- Sys.time()
se_analytical <- std_errors(theta)
t2 <- Sys.time()

# Time taken by Apollo (seconds):
model[["timePost"]]

## [1] 362.3175

# Time taken by the analytical formula (seconds):
as.numeric(t2-t1)

## [1] 1.509144

# For a simple comparison of the validity of the results. See the standard errors obtained by Apollo
unnname(model[["se"]])

## [1] 0.20919880 0.31359040 0.32189578 0.18883824 0.20106642 0.06175350
## [7] 0.06114593 0.06610220 0.04618090 0.04092484 0.03985889 0.04208742
## [13] 0.05077231 0.02503876 0.02254635 0.06387755 0.08474664 0.08237895
## [19] 0.04408279 0.04270831 0.37384477 0.98668916 0.91774191 0.54744840
## [25] 0.57082647 0.33392215 1.00199131 0.90889316 0.51398477 0.53067634
## [31] 0.71812055 0.83634857 0.91279032 0.73747804 0.73713203

# Then, see the analytical standard errors:
se_analytical[["se"]]

## [1] 0.20919880 0.31359041 0.32189578 0.18883824 0.20106642 0.06175350
## [7] 0.06114593 0.06610220 0.04618090 0.04092484 0.03985889 0.04208742
## [13] 0.05077231 0.02503876 0.02254635 0.06387755 0.08474664 0.08237895
## [19] 0.04408279 0.04270831 0.37384477 0.98668917 0.91774191 0.54744840
## [25] 0.57082647 0.33392215 1.00199132 0.90889317 0.51398477 0.53067634
## [31] 0.71812056 0.83634857 0.91279032 0.73747804 0.73713203

```